Open Access | https://doi.org/10.25080/tujc9326



SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

From Hello World to Hello LLM: A Python Developer's Survival Guide

Jasmine Omeke[1], and **Audrey Webb**[2]

[1]Airbnb, [2]Autodesk

Abstract

AI tooling is moving fast, but many Python developers are unsure where to start, or how today's patterns fit into systems they already know how to build. This talk gives a practical, hands-on overview of modern large language model (LLM) development patterns in Python, focused on what you need to know to go from zero to hero. We walk through real-world coding examples from a social-good hackathon, broken into parts that show the core building blocks of modern LLM applications and explain when each pattern makes sense. These examples are designed to require no prior machine learning experience. Attendees will leave understanding how LLM systems work, what problems they solve well, and how to maintain and observe what they build. We cover the modern LLM stack in Python. This includes common patterns such as prompts, function calling, retrieval-augmented generation (RAG), and simple agents. Additionally, we will show how to choose between a script, an agent, and a service. We also cover how to get something working quickly without sacrificing reliability or safety. We share practical guardrails for handling errors and protecting data. And we show how to stand up common workflows, from LLM-powered scripts to lightweight agents and Model Context Protocol-style (MCP-style) services. Attendees will leave with a clear map of the LLM landscape, working Python patterns they can reuse immediately, and the confidence to start building LLM features without a machine learning background.

Keywords LLM, Python, RAG, AI tools, education

Published Jul 13, 2026

Correspondence to
Jasmine Omeke
j.omeke.consults@gmail.com

Open Access

Copyright © 2026 Omeke & Webb. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. What We Learned Building AI Tools with Python

Most Python developers already know how to build real systems, APIs, scripts, services, and pipelines. During a social impact AI hackathon, we applied those same engineering skills to rapidly build AI-powered tools that helped educational institutions become more comfortable using AI to improve student success outcomes.

Like many developers entering the AI space, we had to quickly navigate a growing ecosystem of frameworks, APIs, vector databases, and agent tooling while still shipping working software under tight time constraints.

Here we share the lessons learned, tooling decisions, and implementation patterns that helped us move from experimentation to working applications.

First, let's break the component of AI usage down into familiar engineering concepts:

- validation
- retrieval
- orchestration
- structuring inputs
- structured outputs

We'll walk through:

- the core concepts behind modern AI applications
- common Python libraries and tooling
- reusable code snippets and implementation patterns
- understanding how and when to use these patterns in real systems

2. The Shift: AI as a System Component Instead of Our Base Tool

The biggest shift here is the greater accessibility for anyone to build. You no longer need to train models yourself as modern large language model (LLM) APIs, embedding models, and vector databases are available with minimal setup. We view AI as not replacing your application stack, but another component to be added to existing systems that are useful for ambiguity, language, reasoning, and search. Think of an LLM like a non-deterministic but powerful external API that needs structure, validation, retries, and observability.

AI collapses the gap between expertise and execution. In our hackathon, we paired technologists (who bring architecture, LLM knowledge, data systems, and engineering experience) with education experts (who bring deep context, student needs, institutional knowledge, and on-the-ground experience). Together, through a modular LLM stack (prompts, tools, retrieval, embeddings, vector databases, and guardrails), we delivered working solutions in three days.



Figure 1. *Collaboration during the hackathon.*

3. The Modern AI Stack in Python

Most AI-powered Python applications are assembled from a small set of reusable components:

LLM APIs: ollama-python, litellm, transformers, vllm, llama-cpp-python

Embedding models: sentence-transformers, transformers, InstructorEmbedding, FlagEmbedding (BGE models)

Vector databases: faiss, chromadb, qdrant-client, weaviate-client, milvus

Prompt orchestration: LangGraph, LangChain, Haystack, PydanticAI, CrewAI

Structured output validation: Pydantic, Instructor, Guardrails AI, jsonschema

Tool calling: PydanticAI, LangGraph, LangChain, smolagents, CrewAI

The key takeaway: You are still writing Python. LLMs are another system dependency your application coordinates and manages. We'll show how these pieces connect in practice using code snippets and examples from our social impact hackathon project.

4. Key Terms

For our purposes, during the hackathon, we used the following concepts:

- **LLM:** Generates text from prompts
- **Embeddings:** Turn words or sentences into numbers that capture meaning. Things with similar meanings end up close together ("dog" ≈ "puppy", "pizza" ≈ "burger"). That's how AI can find related ideas, not just exact word matches.
- **RAG:** Retrieve context before generating answers
- **Agent:** A loop where the model decides which tools to call, in what order, based on intermediate results

These concepts cover most real-world AI applications.

5. Requirements Gathering Before Hacking

Before we began using AI, we spent time understanding what our stakeholders considered a good result.

Requirements gathering was an important step to help us understand the goals of our educational institutions. This involved journey mapping. Before building, we worked with educational staff to journey-map their workflows, questions, and pain points. That process helped define:

- What kinds of questions the system should answer
- What data it was allowed to access
- What topics were out of scope
- How responses should be phrased

Once we documented the needs and criteria our stakeholders (educational institutions and staff) had for success, we could begin development work. With the requirements distilled from our clients with this method of planning, we were able to identify LLM patterns for our respective institution's use case. With that in mind, we'll explore patterns of LLM usage with hands-on examples.

6. Core Pattern 1: System Prompts + Data Dictionaries

Jasmine's team used system prompts and a data dictionary to ground the LLM in institutional data and terminology. This created a control layer between user questions, language, and underlying datasets.

Key components:

- **System prompt** defines rules, scope, and response behavior
- **Data dictionary** defines approved institutional terms and metrics
- Together, they constrain interpretation and reduce hallucinations
- Ensures data is used at the correct grain and in the correct context

Jasmine's hackathon project involved building a chatbot to help higher education staff understand a pivotal student success dataset. The university had an institutional research department that was overwhelmed with data requests that took days to answer. AI was an opportunity to make this self-service, but institutions were concerned about data privacy and security. To address that, the chatbot was scoped to return **aggregate data only**, with no individual student-level information exposed.

Prompting became the primary way we defined system behavior, privacy, and guardrails. We created a data dictionary of approved educational terms and concepts to guide the LLM toward domain-specific language and reduce hallucinations. This was important because despite datasets having overlapping subject matter, there was nuance in how data could be used. Some tables contained individual student data (PII) and could not be used at all for a chatbot serving data to internal stakeholders.

This dictionary acted as scaffolding for the LLM:

- Defining the language of the domain
- Clarifying what concepts existed
- Constraining how questions should be interpreted
- Helping the model distinguish valid requests from unsupported ones

Rather than relying on the model's general knowledge, we used structured context to shape how it reasoned about the problem space.

```
SYSTEM_PROMPT = """
You are an educational analytics assistant.

### brief data dictionary ###
Use only the approved terms below:
- Attendance Rate
- Engagement Score
- Intervention Tier
- Student Success Trend

Rules:
- Only return aggregate data
- No individual student data (PII)
- Say "out of scope" if not supported
- Don't join the Attendance Rate data to an external, aggregate student metric table;
  that data contains information at an opposing grain.
"""

response = llm.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": "Which tiers improved attendance?"}
    ]
)
```

Key takeaway: System prompts and data dictionaries turn language into a controlled interface for data.

7. Core Pattern 2: Context Retrieval for Grounding (Database-Backed, RAG-Style)

We extended the data dictionary into a live retrieval layer connected to institutional data systems.

Why it matters:

- Aligns with existing infrastructure

- Bridges prompts with real data systems
- Reduces hallucinations through grounding
- Guides the chatbot toward approved terminology
- Retrieves relevant institutional definitions and resources at query time
- Constrains answers to known educational concepts

Jasmine's hackathon project relied on retrieval to ground responses in a trusted educational context, always at the aggregate level described in Pattern 1. In production, this kind of grounding is often implemented with embeddings and a vector store; under hackathon time constraints, we instead built a simplified, database-backed version of the same idea: a keyword check followed by a direct SQL lookup. Both approaches serve the same purpose, retrieving relevant, approved context before the LLM generates an answer, and both helped guide the chatbot toward approved terminology and constrain it to known concepts. We call this out explicitly here because it's a lighter-weight stand-in for full embedding-based RAG, not a complete RAG pipeline.

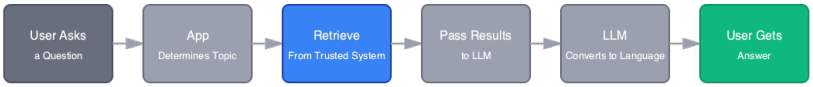


Figure 2. *RAG pipeline flow diagram.*

```
# Data sources (swappable, secured)
DATA_SOURCES = {
    "sql": "...Azure SQL / AAD auth..."
}

# Simplified retrieval layer: a keyword check + direct SQL lookup.
# In production, this step would typically embed the query and
# search a vector store; here we use a lighter database-backed
# lookup to illustrate the same grounding principle.
def fetch_metrics(query):
    if "retention" in query:
        return sql("""
            SELECT metric, value, definition
            FROM student_metrics
            WHERE metric = 'retention'
        """)
    return None

# LLM layer (swappable)
def call_llm(prompt, model="gpt-4o"):
    return llm.chat(model=model, messages=[
        {"role": "system", "content": "Use only provided context."},
        {"role": "user", "content": prompt}
    ])

# App flow
def answer(query, model="gpt-4o"):
    context = fetch_metrics(query)
    prompt = f"{query}\n\nContext:\n{context}"
    response = call_llm(prompt, model=model)
    log = {"query": query, "model": model}
    return response, log
```

Key takeaway: Database-backed context retrieval, a simplified stand-in for full embedding-based RAG, connects the LLM to live institutional data systems and approved terminology.

8. Core Pattern 3: Function Calling & Controlled Output

Not every problem requires an LLM. For numeric forecasts, risk scoring, and trend analysis, the model should not invent results. It should request them from backend systems that can compute them deterministically.

Audrey's approach: She used requirements gathering and journey mapping to understand the goals of her institutions. That research helped her design a predictive analytics dashboard that separated language understanding from analysis workflows, so the LLM acted as an interface layer rather than generating predictions directly. This dashboard was built for authorized advisor use, a different audience and access scope than Jasmine's aggregate-only chatbot, so the two systems apply different levels of data granularity appropriate to their users.

In practice, the chatbot received structured requests, passed them to backend analytics, and then explained the results back to users in clear language. The dashboard provided:

- Retention prediction
- Risk scores per student
- Probability of dropping out
- Attendance trend forecasts
- Credential type prediction
- Gateway course success prediction
- GPA prediction

Key principles:

1. **Separate Understanding from Computation** - The LLM interprets the user's intent, but the predictive model and analytics pipeline do the actual math. This keeps forecasting logic outside the model and makes the system easier to test and trust.
2. **Let the LLM Request Structured Data** - Instead of free-form answers, the LLM calls a function like `get_retention_risk(school_id)`. That request is precise, typed, and limited to the data the backend is designed to return.
3. **Return Predictable Analytics** - The Python system aggregates predictions and returns structured outputs such as risk scores, dropout probability, and attendance trends. The LLM then explains those results in plain language.
4. **Keep It Reproducible and Auditable** - Because predictions are computed outside the model, the same input produces the same output. That makes the system easier to audit, version, and debug, especially in a hackathon setting where you need reliable behavior fast.

```python
from openai import OpenAI

client = OpenAI()

# Predictive model wrapped as an approved function.
# This function returns a school-level average, so the
# question posed to the LLM should match that grain.
def get_retention_risk(school_id):
    students = feature_store.get_students(school_id)
    risk_scores = retention_model.predict_proba(students)
    return {
        "school_id": school_id,
        "avg_retention_risk": float(risk_scores.mean())
    }

tools = [
    {
        "type": "function",
        "function": {
            "name": "get_retention_risk",
            "description": "Retrieve student retention risk metrics",
            "parameters": {
                "type": "object",
                "properties": {
                    "school_id": {"type": "string"}
                },
                "required": ["school_id"]
            }
        }
    }
]

user_question = (
    "What is the average retention risk for School_123 this semester?"
)

# Step 1: LLM determines what analytics are needed
response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {
            "role": "system",
            "content": (
                "Never generate predictions yourself. "
                "Use available analytics functions and explain results."
            )
        },
        {"role": "user", "content": user_question}
    ],
    tools=tools
)

# Step 2: Backend executes predictive model
analytics_result = get_retention_risk(
    school_id="School_123"
)

# Step 3: LLM explains results
final_response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "user", "content": user_question},
        {
            "role": "tool",
            "tool_call_id": response.choices[0].message.tool_calls[0].id,
            "content": str(analytics_result)
        }
    ]
)

print(final_response.choices[0].message.content)
```

This pattern ensured predictions were:

- Reproducible
- Auditable
- Computed separately from the LLM
- Token usage was optimized

Key takeaway: Separate language understanding from computation. Let the LLM be an orchestrator, not a calculator, and keep the question you send it aligned with the grain of data the function actually returns.

9. Core Pattern 4: Simple Agent Loops

While Core Pattern 3 focused on a single function call, many institutional questions in Audrey's project required chaining multiple tools together, and the right tool to call next often depended on what the previous tool returned. This is what separates an agent from a script: instead of Python deciding the order of operations up front, the LLM sees each tool result and decides what to do next, including when it has enough information to stop and answer.

A simple agent loop looks like this:

1. Send the conversation, including any tool results so far, to the LLM
2. If the LLM responds with a tool call, execute that tool in Python and append the result to the conversation
3. Repeat until the LLM responds with a final answer instead of a tool call

Consistent with the advisor-facing scope described in Pattern 3, this agent works with cohort-level and segment-level data rather than naming individual students.

```
tools = [
    get_student_cohort_tool,
    get_retention_risk_tool,
    compare_to_previous_term_tool
]

def execute_tool(name, arguments):
    if name == "get_student_cohort":
        return get_student_cohort(**arguments)
    if name == "get_retention_risk":
        return get_retention_risk(**arguments)
    if name == "compare_to_previous_term":
        return compare_to_previous_term(**arguments)
    raise ValueError(f"Unknown tool: {name}")

messages = [
    {
        "role": "system",
        "content": (
            "You are a retention analytics assistant for university advisors. "
            "Use the available tools to gather whatever data you need before "
            "answering. Report findings at the cohort or segment level only, "
            "never by naming individual students."
        )
    },
    {
        "role": "user",
        "content": (
            "Which first-year student segments carry the most retention risk, "
            "and is risk improving or declining relative to last term?"
        )
    }
]

# Agent loop: the LLM decides which tool to call next, based on what
# earlier tool calls returned, until it decides it has enough to answer.
while True:
    response = client.chat.completions.create(
        model="gpt-4o",
        messages=messages,
        tools=tools
    )
    choice = response.choices[0].message

    if not choice.tool_calls:
        print(choice.content)
        break

    messages.append(choice)

    for tool_call in choice.tool_calls:
        result = execute_tool(
            tool_call.function.name,
            json.loads(tool_call.function.arguments)
        )
        messages.append({
            "role": "tool",
            "tool_call_id": tool_call.id,
            "content": str(result)
        })
```

In a run like this, the model might first call `get_student_cohort` to pull the first-year class, then call `get_retention_risk` on that cohort, then decide on its own whether `compare_to_previous_term` is actually needed to answer the question, or call it again with a narrower segment if the first risk scores looked uneven across groups. None of that branching is written in Python. It comes from the LLM reading each tool result and choosing the next step.

Key takeaway: In a true agent loop, the model decides which tool to call next based on intermediate results, and the loop only ends when the model itself decides it has enough information to answer.

10. LLM Guardrails We Deployed

The goal is not to rely solely on the model, but to build systems that remain reliable even when it makes mistakes.

Because we had two different use cases (Audrey's predictive analytics tool and Jasmine's institutional data chatbot), we deployed different guardrails tailored to each system's risks and requirements.

10.1. Audrey's Project: Predictive Analytics Tool

- **Input Sanitization:** Validate and clean user inputs before they reach the model
- **Sensitive Data Handling:** Never send PII, secrets, or sensitive data to an LLM without explicit review
- **Structured Output Validation:** Never use raw LLM output without validation; enforce schemas before downstream systems
- **Prompt Logging & Observability:** Log prompts, responses, latency, token counts, and tool calls to detect degradation early
- **Retries & Timeouts:** Set explicit timeouts on LLM API calls; use retries with exponential backoff for transient failures

Problems to be Solved	Guardrail
Malicious input	Input Sanitization
Data exposure risk	Sensitive Data Handling
Invalid outputs break systems	Structured Output Validation
Silent failures	Prompt Logging & Observability
API failures	Retries & Timeouts

10.2. Jasmine's Project: Institutional Data Chatbot

- **Hallucination Safeguards:** Ground factual answers in verified data; require citations when needed; allow the model to say "I don't know"
- **Detecting Off-Topic Conversations:** Identify and redirect conversations that drift too far from intended scope
- **Rejecting Unsupported Requests / Escalating Sensitive Queries:** Redirect unsupported requests to the right workflow; escalate sensitive queries that need policy checks or special handling
- **Dynamic Retrieval Strategy Selection:** Dynamically select retrieval strategies based on topic, intent, and confidence in the answer

Problems to be Solved	Guardrail
LLM invents facts	Hallucination Safeguards
Off-topic requests	Detecting Off-Topic Conversations
Out-of-scope questions	Rejecting Unsupported Requests / Escalating Sensitive Queries
Wrong retrieval strategy	Dynamic Retrieval Strategy Selection

11. Building the Right Complexity for the Problem

Not every problem needs an agent. Use:

- A single prompt for simple classification or extraction
- A pipeline for fixed workflows
- An agent only when adaptive multi-step reasoning is required
- Function calling when computation must be deterministic
- RAG when you need to ground answers in external knowledge

Start with the simplest working approach before adding complexity. The path is straightforward:

1. Start with a simple LLM API call
2. Add structured outputs
3. Add tool calling for focused, useful actions
4. Add retrieval-based grounding when external knowledge matters
5. Introduce agent loops only when the task genuinely requires the model to decide what to do next

12. Conclusion

You do not need deep ML expertise to build useful AI systems. You need:

- Strong engineering fundamentals
- Clear abstractions
- Careful system design that reduces cost, latency, and failure modes
- Start small, observe everything, and expand with purpose

The core lesson from our hackathon: The path forward is straightforward. Build on solid engineering basics, add capability in layers, and use the simplest abstraction that solves the problem.

In three days, we went from exploring requirements to shipping working tools because we treated AI like any other system dependency with respect for its probabilistic nature, careful validation, and thoughtful orchestration. Python developers already know how to do this. AI is just another component in your stack.

13. GenAI Usage Disclosure

Generative AI (Claude, Anthropic) was used as an editing aid for this paper, primarily to help tighten sentence structure and shorten overly long sentences in the abstract, and to check grammar and flow throughout. All technical content, code examples, patterns, and conclusions reflect the authors' own work and hackathon experience; no generative AI was used to produce the underlying ideas, code, or results described in this paper.